

A Phase-Aware Adaptive Bank Mapping Approach for Reducing GPU Shared Memory Bank Conflicts

Devaananth P
Department of Computer Science and Engineering
SRM Institute of Science and Technology
Chennai, India
devaananthmsd31@gmail.com

Abstract—This paper discusses a phase-aware adaptive bank mapping policy that dynamically adjusts the shared memory mapping based on the current access pattern. In shared-memory programming, bank conflicts can reduce program efficiency when multiple threads access the same memory bank simultaneously. The proposed solution uses lookahead information, hybrid decision-making, a causal watchdog, and safe remapping boundaries to prevent unnecessary or incorrect remapping. The experimental evaluations included various synthetic stress tests and access patterns derived from kernels, such as matrix transpose, parallel reduction, and prefix scan. It has been shown that the suggested method can achieve the same level of conflict-cost minimization as the optimal static mapping if one of the mappings is optimal for the entire phase, and it can significantly reduce conflict cost for workloads with changing access patterns across phases. It has also been demonstrated that remapping should be done only within safe data-lifetime boundaries, and any deferred decisions must be re-verified before implementation.

Keywords— GPU shared memory, bank conflicts, adaptive bank mapping, phase-aware optimization, runtime remapping, CUDA

I. INTRODUCTION

Graphics Processing Units are made to do a lot of work at once. Instead of finishing one big task from beginning to end they take on many small tasks all at the same time. This is why they are good for tasks like graphics, simulations and programs that need a lot of data. To make these tasks go fast GPUs use memory. Shared memory is much quicker to reach than memory. This speed only works when threads access it in a smart way. Shared memory is split into banks. You can imagine them as checkout counters in a store. If each thread goes to a bank the access can happen all at once. If several threads try to use the same bank at the same time they have to wait. This is called a bank conflict. It makes things slower because some requests have to be handled one after another. There are already ways to deal with these conflicts. Padding is one method. Changing how data is stored in memory can also help. Other methods change how addresses are linked to banks. One example is using XOR-based mappings. These can work well when the way data is accessed stays the same. The problem is that some tasks do not follow this pattern. A mapping that works in one part of the program might be bad in the next part. This research looks at this changing behavior. Instead of using one fixed way to map addresses to banks for the whole time the program runs the new method can switch between mappings based on how the data is being accessed. It uses information about what's coming next a mix of decision rules a watchful system and safe points to change the mapping. The main goal is not just to reduce conflicts but to do it without causing any errors. So changing the mapping is only allowed when the old shared-memory data is no longer being

used. The method is tested with made-up tasks that change over time as well as real tasks, like matrix transpose, parallel reduction and prefix scan.

II. RELATED WORK

Some techniques aim to hide the delay that bank conflicts create while others work on altering how addresses are mapped to memory banks. Gou and Gaydadjiev[1] came up with an elastic pipeline method that reduces the impact of bank conflicts while a program runs. Their study mainly tackles the delay from bank conflicts, not the mapping itself. Another key direction is XOR-based bank mapping. Van den Braak et al[2] examined XOR hash functions for GPU scratchpad memories. The goal is to spread accesses across banks by changing which address bits pick a bank. Their experiments showed that this mapping can cut bank conflicts a lot when the access patterns fit the design. The research similar to this paper comes from Han et al[3] who suggested an access-pattern-based adaptive mapping for GPGPU scratchpad memory. Their method gathers access data while the first thread block runs, then **selects** a mapping for later execution. This shows that adaptive bank mapping can help. The chosen mapping relies mainly on past behavior. What sets this work apart is its focus. Rather than assuming a single mapping stays good after the first decision the new method accepts that access patterns may shift during execution. It also introduces remapping points to keep the mapping steady while old shared-memory data stays active. Deferred decisions are re-examined before use helping to avoid a mapping that no longer fits the access pattern.

III. PROPOSED METHODOLOGY

The new method changes the way the shared memory bank is mapped depending on what's happening right now. Instead of using the same mapping all the time, it adapts. Three different ways of mapping are looked at in this project: normal mapping and two different XOR-based mappings. The controller looks at how each of these would handle the steps and then picks the one that is likely to have fewer problems with memory banks. The choice also uses lookahead information. A mix of methods is used so the controller doesn't switch between mappings too much. A special part of the system called a watchdog looks at what has happened recently. It helps fix things if the current mapping starts to perform poorly. There is also a way to handle things if the way the memory is being used changes very fast and the controller needs to act quickly. One key part of the system is making sure the changes to the mapping are safe. If you change the way the memory is mapped while old data is still being used it can cause incorrect reads. For this reason a change is only allowed at points when the old data is no longer needed. If a change is not possible now it is saved and checked again later when it is

safe to do so. This check is important because a decision that is delayed might not be useful anymore. The controller checks what is happening now before making the change. This helps make sure the change is both helpful and correct.

IV. EXPERIMENTAL SETUP

The proposed method was tested using a simulator that was built for this project. The model uses 32 shared-memory banks and 32 threads in a warp. Three mapping methods were compared: normal mapping, XOR-A and XOR-B. The main measurement used in the experiments is conflict cost, which **measures** the highest number of threads accessing the same bank during a memory operation. Two types of workloads were used. The first group includes workloads with different access patterns such as stable, irregular, clustered and phase-changing patterns. These were helpful for seeing how the controller handles memory access changes during execution. The second group uses kernel-derived access patterns taken from GPU operations. These include matrix transpose, parallel reduction and prefix scan. A separate phase-mixed workload was also created using stride patterns so that the controller could be tested under changing phases. For the safe-remapping experiments mapping changes were only allowed when the previous shared-memory data was no longer active. The adaptive controller was then compared with the **best** static mapping for each workload. The number of mapping switches, conflict cost and unsafe remapping attempts were recorded during evaluation. The source code and validation scripts used in this work are available at: <https://github.com/Devaanath357/phase-aware-gpu-bank-mapping>.

V. RESULTS AND DISCUSSION

The final experiments showed that the adaptive method behaves depending on the type of workload. For matrix transpose, parallel reduction and prefix scan one static mapping was already good for the run. In these cases the adaptive controller matched the static mapping and did not create any unsafe remapping. The biggest improvement appeared in the phase-mixed workload, where the access pattern changes between phases. The best static mapping had a conflict cost of 3296. The unrestricted adaptive controller lowered this to 2353 while the lifetime-safe adaptive controller lowered it more to 1559. This is a 52.70 percent reduction in conflict cost compared with the static mapping. The safe controller also finished the phase-mixed test with zero unsafe remapping attempts. This shows that the controller can still adapt even when mapping changes are limited to data-lifetime boundaries. Another important result was that delayed mapping decisions had to be checked before being used because an old decision could become useless if the access pattern changed before the next safe point. Overall the results show that adaptive bank mapping is most useful when the workload changes its shared-memory access pattern during execution. When one mapping is already good, for the workload the adaptive bank mapping mainly helps by avoiding unnecessary changes.

Benchmark	Best Static	Unrestricted Adaptive	Lifetime-safe Adaptive
Matrix Transpose	128	128	128
Parallel Reduction	768	768	768
Prefix Scan	1536	1536	1536
Phase-Mixed	3296	2353	1559

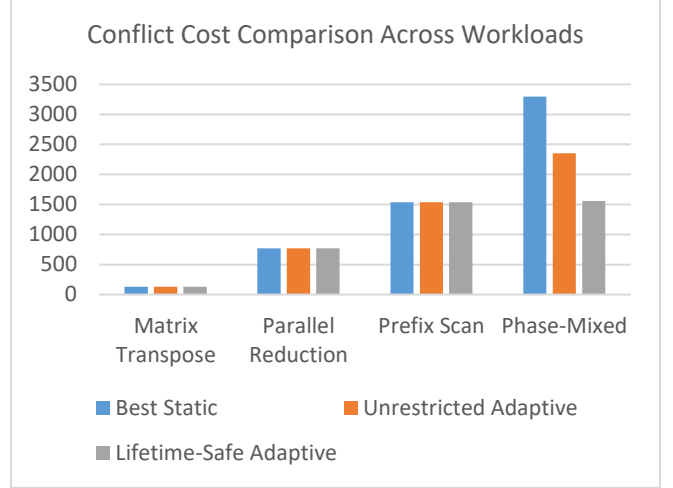


Fig. 1. Conflict cost comparison between static and adaptive bank mapping methods.

VI. LIMITATIONS

The results in this work come from a simulator and from conflict cost not from GPU execution time. The XOR-A and XOR-B mappings used in the experiments are simplified mapping models. They do not match exactly the mapping inside a real GPU. The kernel-derived tests also have limits. Matrix transpose, reduction and prefix scan were used to create realistic access patterns but they remain trace-based experiments instead of direct measurements from GPU hardware. Another limitation is that safe remapping depends on knowing when old shared-memory data is no longer being used. In this work these safe points are defined by the lifetime of the shared-memory region. More work would be needed to detect these points automatically in a GPU system. The phase-mixed workload produced the improvement but it remains a synthetic workload built from realistic access patterns. Because of this the 52.70 percent reduction should be understood as a result from the experimental model, not as a guaranteed speedup on real hardware.

VII. CONCLUSION

This work introduced a phase-aware adaptive bank mapping method for GPU shared memory. The key idea is to not stick to one fixed mapping for the program run. Instead it changes the mapping when the way memory is accessed shifts. Experiments showed that the method performs as well as the best static mapping when one mapping works well for the whole workload. In cases where the workload has phases the lifetime-safe adaptive controller cut conflict costs by 52.70 percent compared to the best static mapping. It also avoided unsafe remapping, which is important for correctness. The results also highlighted that remapping and rechecking decisions before acting are key to keeping the method

working right. Even though the study used simulation and trace-based testing it shows that phase-aware adaptive bank mapping could be an approach for workloads where shared-memory access patterns change over time.

REFERENCES

- [1] C. Gou and G. N. Gaydadjiev, “Addressing GPU on-chip shared memory bank conflicts using elastic pipeline,” *International Journal of Parallel Programming*, vol. 41, pp. 400–429, 2013.
- [2] G.-J. van den Braak, J. Gómez-Luna, J. M. González-Linares, H. Corporaal, and N. Guil, “Configurable XOR hash functions for banked scratchpad memories in GPUs,” *IEEE Transactions on Computers*, vol. 65, no. 7, pp. 2045–2058, Jul. 2016, doi: 10.1109/TC.2015.2479595.
- [3] F. Han, L. Li, K. Wang, F. Feng, H. Pan, J. Sha, and J. Lin, “An access pattern based adaptive mapping function for GPGPU scratchpad memory,” *IEICE Electronics Express*, vol. 14, no. 12, Art. no. 20170373, 2017.
- [4] A. Horga, A. Rezine, S. Chattopadhyay, P. Eles, and Z. Peng, “Symbolic identification of shared memory based bank conflicts for GPUs,” *Journal of Systems Architecture*, vol. 127, Art. no. 102518, Jun. 2022, doi: 10.1016/j.sysarc.2022.102518.